BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE

B.Sc. Computer Science · Capstone Project · Academic Year 2025–2026

USER MANUAL

Solana Local Explorer

Everything you need to run, use and troubleshoot a self-hosted block explorer for a private Solana validator.

AUTHORS

Garvit Dadheech · Shubh Kesharwani

SUPERVISOR

Dharma Teja

THE 30-SECOND VERSION

Install Docker. Clone the repository. Run `docker compose up --build -d`. Open `http://localhost:3000`. That is the whole procedure.

Version 1.0 · 28 August 2026 · github.com/shubhiscoding/Solana-Local-Indexer-Explorer

CONTENTS

1. What This Tool Does 2
2. Before You Start 3
3. Starting the Explorer 4
4. Using the Interface 6
   - 4.1 Dashboard 6
   - 4.2 Transaction list 7
   - 4.3 Transaction detail 8
   - 4.4 Account detail and balance filtering 9
   - 4.5 Global search and theme 9
5. Using the REST API 11
6. Common Tasks 13
7. Configuration 15
8. Troubleshooting 17
9. Frequently Asked Questions 18
10. Glossary 19

Quick reference card

TO DO THIS	RUN THIS
Start everything (first time)	`docker compose up --build -d`
Start everything (afterwards)	`docker compose up -d`
Check that all four services are up	`docker compose ps`
Watch what is being indexed	`docker compose logs -f indexer`
Stop, keeping your data	`docker compose down`
Stop and wipe everything	`docker compose down -v`
Open the explorer	`http://localhost:3000`
Talk to the validator	`http://localhost:8899`
Query the database directly	`psql postgresql://postgres:password@localhost:5433/explorer`

CHAPTER 1

What This Tool Does

Solana Local Explorer gives you a working block explorer for a Solana chain that no public explorer can see — typically the `solana-test-validator` running on your own machine.

It starts a local Solana validator with the Yellowstone Geyser plugin attached, streams every transaction the validator produces into a PostgreSQL database, and serves a web interface and a REST API over that data. You get the things you would expect from Solscan or the official Solana Explorer — a live dashboard, browsable transaction history, per-transaction detail with accounts and instructions, and per-account balance history — for a chain that exists only on your laptop.

What you get

- A live dashboard with total transactions, failures, success rate and latest slot, refreshing every five seconds.
- An activity chart showing successes and failures over time, across four ranges.
- A paginated, browsable transaction history.
- Full transaction detail: accounts, instructions, fee, compute units, memos, and program logs for failures.
- Per-account balance history with pre-balance, post-balance and signed net change, and an amount filter.
- Global ⌘K search across signatures and addresses.
- Five REST endpoints for scripting and dashboards.
- The ability to point the same indexer at devnet, testnet or mainnet by setting one environment variable.

What it does not do

- **Decode instructions into named methods.** Instructions appear as a program index and base-58 data. Decoding them would require per-program IDLs.
- **Interpret SPL token or NFT balances.** Balances are tracked in lamports only.
- **Index history from before it started.** Indexing is forward-only from the moment the indexer connects.
- **Submit transactions.** The explorer is strictly read-only with respect to the chain.
- **Authenticate users.** Run it on localhost or a trusted network — anyone who can reach port 3000 can read everything it has indexed.

CHAPTER 2

Before You Start

2.1 Requirements

REQUIREMENT	DETAIL
Docker Desktop	Version 24.x or newer, with Docker Compose **v2**. Check with `docker compose version` — note the space, not a hyphen.
Free RAM	About 4 GB, for the validator, database and two Node services running together.
Free disk	About 4 GB — roughly 2 GB of images plus the validator ledger and the database volume.
Free ports	`3000` (explorer), `8899` (Solana RPC), `10000` (gRPC), `5433` (PostgreSQL).
Operating system	macOS, Linux, or Windows 10/11 with WSL2.
Internet	Needed for the first build only, to pull base images and clone the Geyser plugin source.

> **YOU DO NOT NEED ANY OF THESE INSTALLED**
> Rust, the Solana CLI, Node.js, pnpm or PostgreSQL. Docker builds and runs all of them inside containers. You only need Node and pnpm if you want to develop the code outside Docker (Section 7.4).

2.2 Checking your ports are free

```
lsof -i :3000    # macOS / Linux — repeat for 8899, 10000, 5433

Get-NetTCPConnection -LocalPort 3000    # Windows PowerShell
```

If a command returns a process, either stop it or remap the port in `docker-compose.yml` — for example change `"3000:3000"` to `"3001:3000"` and use `localhost:3001` instead.

2.3 A note for Apple Silicon users

The validator image is pinned to `linux/amd64` and therefore runs under Rosetta emulation on M-series Macs. This is expected and correct. The only visible effect is that the first build is slower; everything afterwards is cached and runs normally.

CHAPTER 3

Starting the Explorer

3.1 The one command

```
git clone https://github.com/shubhiscoding/Solana-Local-Indexer-Explorer.git
cd Solana-Local-Indexer-Explorer
docker compose up --build -d
```

The first build takes several minutes, because the validator image compiles the Yellowstone Geyser plugin from Rust source. This happens once; later starts take seconds. Four services then come up in a fixed order:

1. **validator** and **postgres** start together.
2. Both must report *healthy* before anything else starts.
3. **indexer** starts, applies database migrations, and subscribes to the validator's gRPC stream.
4. **explorer** starts and serves the interface on port 3000.

3.2 Confirming it worked

```
docker compose ps
```

All four rows should show `Up`, with validator and postgres also showing `(healthy)`:

```
NAME                STATUS
explorer-postgres   Up ... (healthy)
solana-explorer     Up ...
solana-indexer      Up ...
solana-validator    Up ... (healthy)
```

Then check that data is actually flowing:

```
docker compose logs indexer --tail 20
```

You are looking for these three things, in order:

LOG LINE	MEANS
`Migrations completed`	The database schema is in place.
`Connecting to Yellowstone gRPC: http://validator:10000`	The indexer resolved its endpoint.
`Subscribed to slots and transactions`	The stream is open. Indexing has begun.
`Indexed transaction 5Kd… (slot 1234)`	Transactions are being written to the database.

Finally, open **`http://localhost:3000`**.

3.3 Generating your first transactions

A fresh test validator produces vote transactions on its own, so the dashboard will not be empty. To create a transaction you control:

```
docker exec solana-validator solana-keygen new \
  -o /root/.config/solana/id.json --no-bip39-passphrase --force

docker exec solana-validator solana airdrop 5 --url http://127.0.0.1:8899

docker exec solana-validator solana transfer --allow-unfunded-recipient \
  11111111111111111111111111111112 0.01 --url http://127.0.0.1:8899
```

Refresh the dashboard: the transaction count increases and the transfer appears in the recent transactions table within about five seconds.

CHAPTER 4

Using the Interface

4.1 Dashboard — /

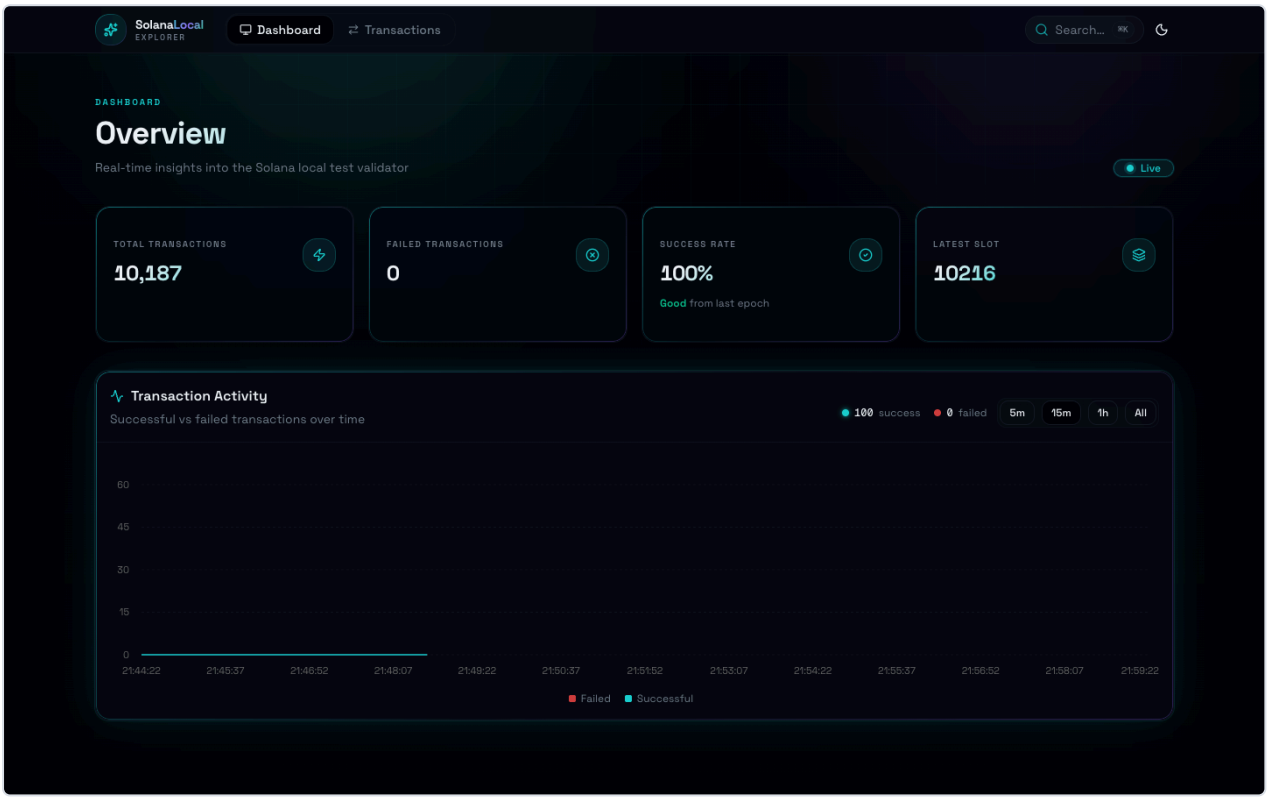


Figure 1 — The dashboard, refreshing every five seconds.

ELEMENT	WHAT IT SHOWS
Total Transactions	Every successful transaction indexed since the database was last reset.
Failed Transactions	Transactions that executed but returned a program error. These are stored separately, with their logs.
Success Rate	Successful ÷ (successful + failed), to one decimal place.
Latest Slot	The highest slot number seen. It should keep climbing while the validator runs — a frozen value means ingestion has stopped.
Transaction Activity	Successes and failures stacked over time. Use the 5 m / 15 m / 1 h / All buttons to change the window; re-bucketing happens instantly in the browser.
Recent Transactions	The ten newest transactions. Hover a truncated signature to see it in full, or use the copy button beside it.

The *Live* badge in the header indicates the five-second refresh is running. If the backend is unreachable you get an explicit error panel rather than a blank page.

4.2 Transaction list — `/transactions`

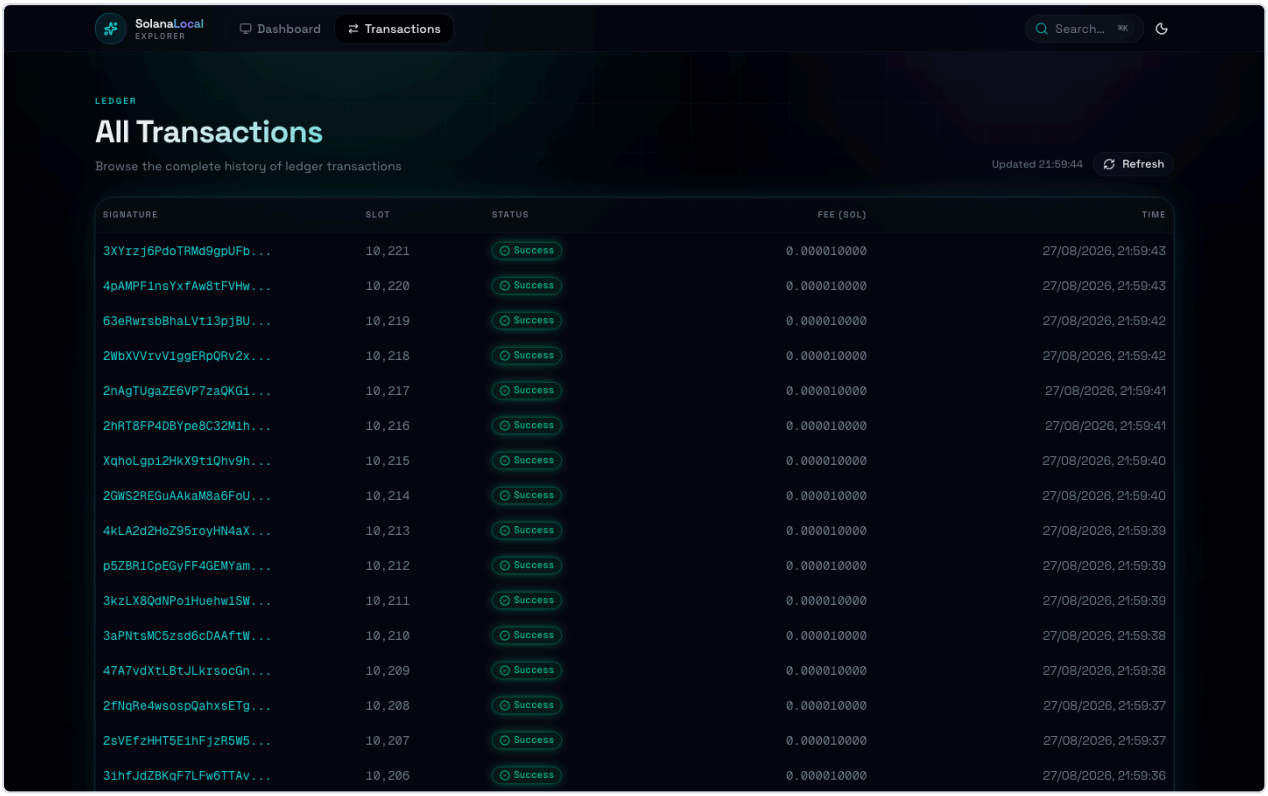


Figure 2 — Paginated transaction browsing.

Every indexed transaction, newest first, twenty per page. The page number lives in the URL, so you can bookmark or share a specific page and it will still be there after a reload. Select any signature to open its detail page.

4.3 Transaction detail — `/transactions/[signature]`

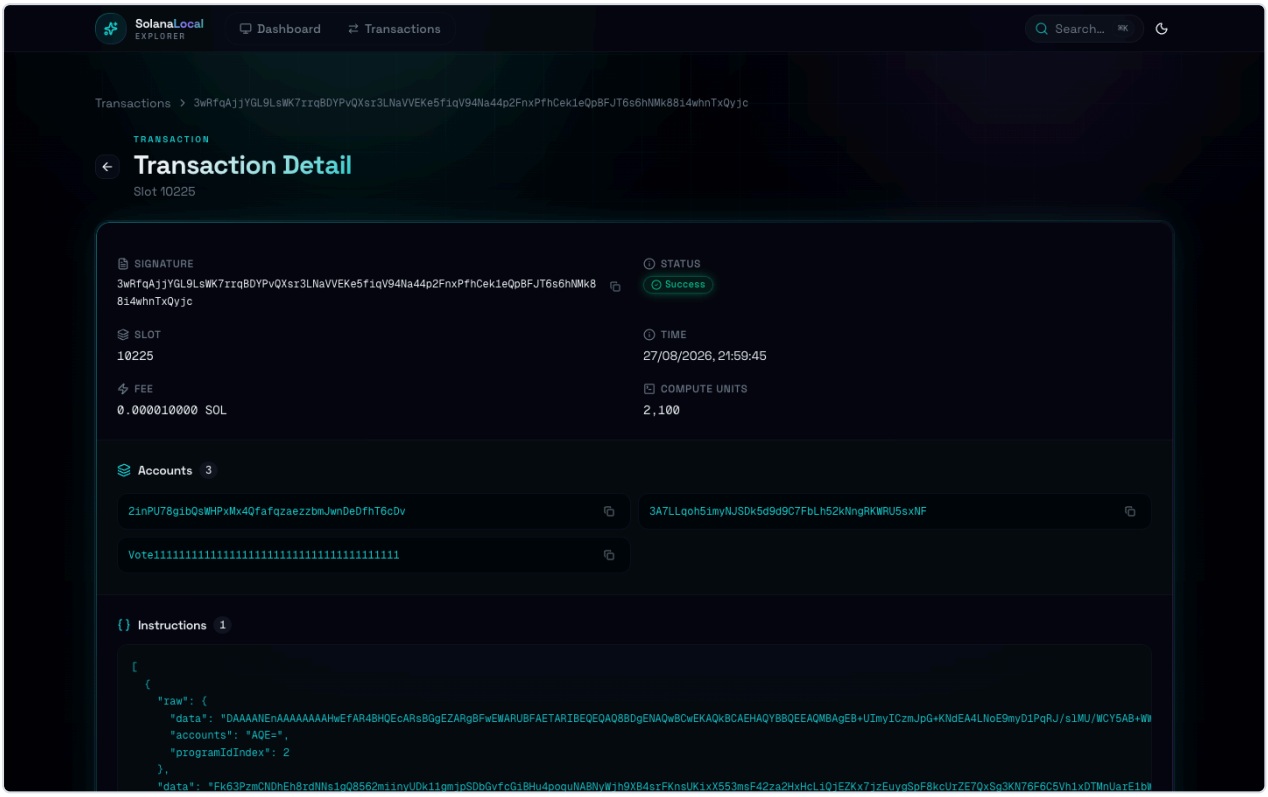


Figure 3 — Full detail for one transaction.

SECTION	CONTENTS
Status	Success or failure, with the slot the transaction landed in.
Fee	Charged in SOL, converted from lamports and shown to nine decimal places.
Compute units	Consumed by execution, where the validator reported it.
Accounts	Every account key, in wire order. Each one links to its account page — this is the usual way to move from a transaction to a balance history.
Instructions	Each instruction's program index, referenced account indices and base-58 data, in execution order.
Memos	Any memo content attached to the transaction.
Error and logs	For failed transactions only: the error returned and the full program log, in a scrollable panel. This is usually the fastest way to see why a program rejected an instruction.

4.4 Account detail and balance filtering — `/accounts/[address]`

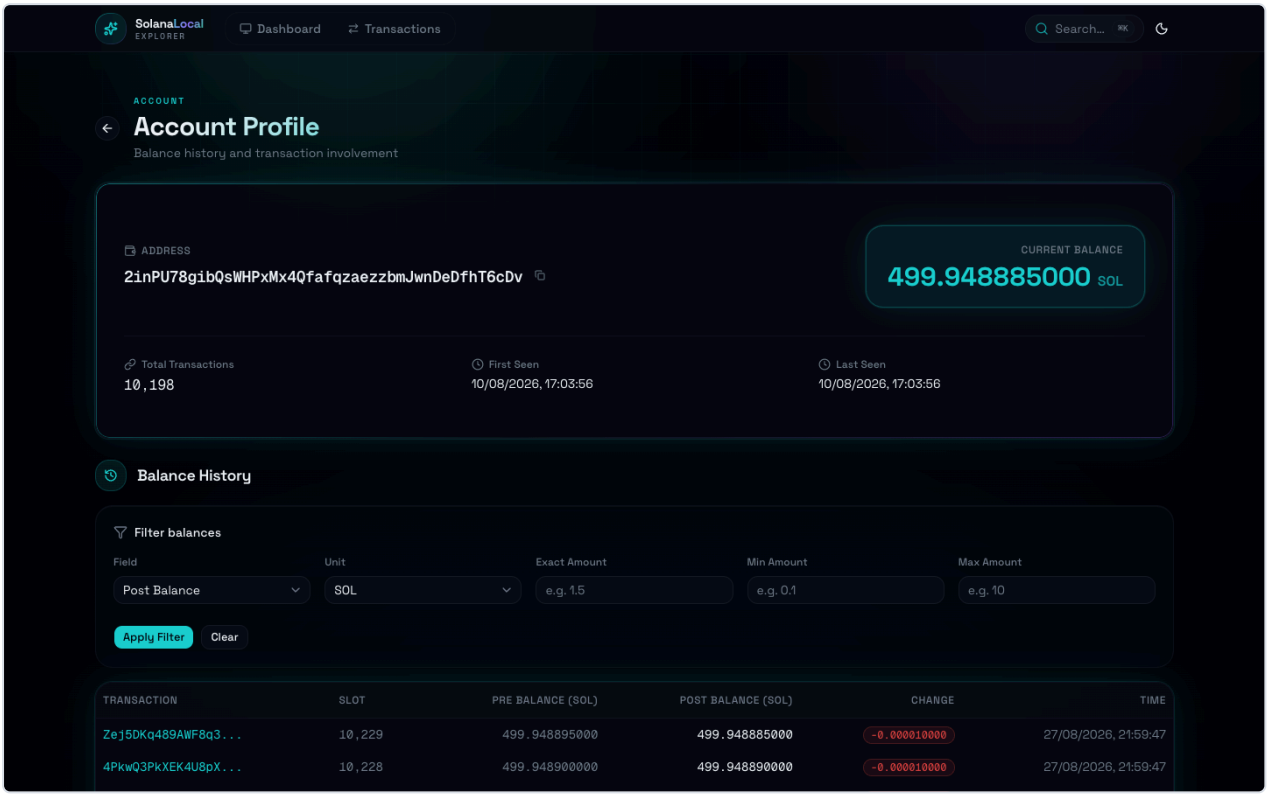


Figure 4 — Balance history with the amount filter.

The header shows the address (with a copy button), when it was first and last seen, and its current balance — taken from the most recent balance row, so it always reflects reality even when a filter is applied. The table below lists one row per transaction the account took part in, with its balance before, its balance after, and the signed change. Gains are shown in the accent colour with a + prefix; losses in the warning colour.

USING THE FILTER

CONTROL	EFFECT
Field	Which column to search: *Post Balance* (default), *Pre Balance* or *Balance Change*.
Unit	Whether your numbers are in *SOL* (default) or raw *lamports*.
Exact Amount	Matches that value exactly. If set, it overrides the min/max fields.
Min / Max Amount	An inclusive range. You may set just one of the two.
Apply Filter / Clear	Applying writes the filter into the URL, so the filtered view is shareable and survives a reload. Clear returns to the unfiltered history.

Worked example: to find every transaction in which this account gained more than 0.5 SOL, set Field to *Balance Change*, Unit to *SOL*, Min Amount to `0.5`, and apply.

4.5 Global search and theme

Press **⌘K** (macOS) or **Ctrl-K** (Windows/Linux), or select the search control in the navigation bar. Type at least **eight characters** of a transaction signature or an account address. Results are grouped by type; selecting one navigates to its detail page.

WHY EIGHT CHARACTERS

Search matches by prefix. A very short prefix would match a large fraction of the table and return noise, so the minimum is enforced by the server as well as the interface. Search does not match text in the middle of an identifier — paste from the start.

The sun/moon control in the navigation bar switches between light, dark and system themes. Your choice persists across reloads. If your operating system requests reduced motion, animations are disabled automatically.

CHAPTER 5

Using the REST API

Everything the interface shows is also available as JSON, so you can script against it or build your own dashboard. All endpoints are `GET` and need no authentication.

> **IMPORTANT: 64-BIT VALUES ARE STRINGS**
>
> Slots, fees, compute units and lamport balances are 64-bit integers. JavaScript's `Number` cannot represent them all exactly, so the API returns them as decimal *strings*. Parse them with a big-integer type — not with `parseInt` — if the value could be large. Timestamps are ISO-8601.

ENDPOINT	RETURNS
`/api/stats`	Dashboard totals, success rate, latest slot and the ten most recent transactions.
`/api/transactions?page=&limit=`	Paginated transaction list, newest first. Limit defaults to 20 and is capped at 100.
`/api/transactions/{signature}`	One transaction in full. Falls back to the failed-transaction record; 404 if neither exists.
`/api/accounts/{address}`	Account record, latest balance, aggregate change and paginated balance history. Accepts the filter parameters below.
`/api/search?q=`	Up to three signature matches and three address matches by prefix. Empty below eight characters.

5.1 Examples

```
# Dashboard totals
curl -s localhost:3000/api/stats | jq

# Second page of ten transactions
curl -s "localhost:3000/api/transactions?page=2&limit=10" | jq '.transactions[].signature'

# One transaction in full
curl -s localhost:3000/api/transactions/<SIGNATURE> | jq

# Balance history where the account gained more than 0.5 SOL
curl -s "localhost:3000/api/accounts/<ADDRESS>?field=balanceChange&minAmount=0.5&unit=sol" | jq

# Same query expressed in raw lamports
curl -s "localhost:3000/api/accounts/<ADDRESS>?
field=balanceChange&minAmount=500000000&unit=lamports" | jq

# Prefix search
curl -s "localhost:3000/api/search?q=5KdAbc12" | jq
```

5.2 Account filter parameters

PARAMETER	VALUES	NOTES
`field`	`postBalance` · `preBalance` · `balanceChange`	Defaults to `postBalance`. Anything unrecognised falls back to the default rather than erroring.

PARAMETER	VALUES	NOTES
`unit`	`sol · lamports`	Defaults to `sol`. SOL values are converted to lamports before comparison.
`amount`	number	Exact match. Takes precedence over the range parameters.
`minAmount`	number	Inclusive lower bound.
`maxAmount`	number	Inclusive upper bound.
`page · limit`	integers	Standard pagination; limit capped at 100.

CHAPTER 6

Common Tasks

6.1 Trace what a transaction did to an account's balance

1. Find the transaction — from the dashboard, the transaction list, or ⌘K search.
2. Open its detail page and locate the account you care about in the accounts list.
3. Select the address to open its account page.
4. Find the row for that transaction: it shows the balance before, the balance after, and the net change.

6.2 Find out why a transaction failed

1. Failed transactions are shown with a failure badge in the transaction list.
2. Open the detail page. The error returned by the runtime appears at the top.
3. Read the program log panel below it — the last few lines before the failure normally name the program and the instruction that rejected.

6.3 Find large balance movements for an account

1. Open the account page.
2. Set Field to *Balance Change* and Unit to *SOL*.
3. Enter a Min Amount and apply. The table, the totals and the pagination all reflect the filter.
4. Copy the URL if you want to share exactly that view.

6.4 Reset and start clean

```
docker compose down -v      # removes the ledger and the database volumes
docker compose up -d        # fresh ledger, empty database, migrations re-applied
```

Use `down` without `-v` if you want to stop the stack but keep everything you have indexed.

6.5 Index a remote chain instead of the local validator

```
YELLOWSTONE_ADDR=your-grpc-host:10000 \
YELLOWSTONE_XTOKEN=your-token \
  docker compose up -d
```

This works against any Yellowstone-compatible endpoint, including commercial devnet, testnet and mainnet providers. The address may be `host:port` or a full `http(s)://` URL. Confirm it took effect in `docker compose logs indexer` — the “Connecting to Yellowstone gRPC” line will show your endpoint.

6.6 Query the database directly

```
psql postgresql://postgres:password@localhost:5433/explorer

-- Ten most recent transactions
SELECT signature, slot, success, fee FROM "Transaction" ORDER BY slot DESC LIMIT 10;

-- Balance rows for one transaction, in account-key order
SELECT "accountIndex", "preBalance", "postBalance", "balanceChange"
FROM "AccountBalance" WHERE "transactionId" = '<id>' ORDER BY "accountIndex";

-- Confirm there are no duplicate signatures
SELECT signature, COUNT(*) FROM "Transaction" GROUP BY signature HAVING COUNT(*) > 1;
```

CHAPTER 7

Configuration

7.1 Environment variables

VARIABLE	SERVICE	DEFAULT	PURPOSE
`YELLOWSTONE_ADDR`	indexer	`validator:10000`	gRPC endpoint to subscribe to. Accepts `host:port` or a full URL.
`YELLOWSTONE_XTOKEN`	indexer	unset	Authentication token, if your endpoint requires one.
`DATABASE_URL`	indexer, explorer	`postgresql://postgres:password@postgres:5432/explorer`	PostgreSQL connection string.

7.2 Ports

Port mappings live in `docker-compose.yml`. The left-hand number is the host port; change it if something else is using it. For example, to serve the explorer on 3001:

```
explorer:
  ports:
    - "3001:3000"
```

7.3 Using an existing ledger

By default the validator writes to a Docker named volume. To use a ledger directory from your host instead, replace the volume with a bind mount:

```
volumes:
  # - validator-ledger:/ledger
  - ./test-ledger:/ledger
```

7.4 Running the code outside Docker

Only needed if you want to iterate on the source. Requires Node.js 20+ and pnpm 10.x.

```
corepack enable && corepack prepare pnpm@10.27.0 --activate

cd explorer
pnpm install
pnpm db:generate    # regenerate the Prisma client after any schema change
pnpm dev            # http://localhost:3000, against the Dockerised database on 5433

cd ../indexer
pnpm install
pnpm dev            # connects to YELLOWSTONE_ADDR, default localhost:10000
```

7.5 Running the tests

```
cd explorer && pnpm install && pnpm test    # 9 serialisation tests
cd indexer  && pnpm install && pnpm test    # 29 gRPC parsing tests
```

Both suites should report all tests passing, in well under a second. Neither needs a running validator or database.

CHAPTER 8

Troubleshooting

SYMPTOM	LIKELY CAUSE	WHAT TO DO
`docker compose up` fails with a port conflict	Something is already using 3000, 8899, 10000 or 5433	Find it with `lsof -i :<port>` and stop it, or remap the port in `docker-compose.yml` (Section 7.2).
`docker compose`: unknown command	Docker Compose V1 (hyphenated) instead of V2	Update Docker Desktop, or install the Compose V2 plugin on Linux.
The validator build hangs or fails during `cargo build`	Unstable network during the plugin clone/build, or too little RAM	Retry `docker compose up --build -d`. Give Docker Desktop at least 4 GB (Settings → Resources).
The indexer container keeps restarting	Validator or database not healthy yet, or a wrong `DATABASE_URL`	`docker compose logs indexer` and `docker compose ps`. Both validator and postgres must show `(healthy)` first.
The explorer loads but shows zero transactions	No traffic yet, or the indexer is not connected	Send a test transaction (Section 3.3), then look for `Indexed transaction …` in the indexer log.
Latest Slot is frozen	The gRPC stream dropped	Check the indexer log for stream errors and `docker restart solana-indexer`. No data is lost — writes are idempotent, so re-indexing is safe.
An account page says "Account not found"	That address has no balance rows — it was last seen before balance tracking existed	Send a new transaction involving the address, or reset with `docker compose down -v`.
⌘K search returns nothing	Fewer than eight characters, or the prefix does not match	Paste at least eight characters from the *start* of the signature or address — search does not match mid-string.
The `solana` CLI on your host cannot connect	Wrong RPC URL configured	Confirm the validator is healthy, then `solana config set --url http://localhost:8899`.
Prisma client errors after pulling new code	The schema changed but the client was not regenerated	`cd explorer && pnpm db:generate`, or rebuild the images.
Everything is slow on an M-series Mac	The validator image runs under Rosetta emulation	Expected on the first build. Later starts use the cache and are much faster.

8.1 Collecting information before asking for help

```
docker compose ps                    # which services are up and healthy
docker compose logs --tail 100       # recent output from everything
docker compose logs indexer --tail 50 # ingestion specifically
docker --version && docker compose version
```

CHAPTER 9

Frequently Asked Questions

Does this work with mainnet?

Yes, if you have access to a Yellowstone-compatible gRPC endpoint for it. Set `YELLOWSTONE_ADDR` and, if required, `YELLOWSTONE_XTOKEN` (Section 6.5). Bear in mind that mainnet volume is vastly higher than a local validator's, and the ingestion path has not been tuned for it — see the limitations in the project report.

Will it show transactions from before I started it?

No. Indexing is forward-only from the moment the indexer connects. Because Docker Compose starts the indexer as soon as the validator is healthy, you will not normally miss anything on a fresh stack. Historical backfill is listed as future work.

Is my data lost when I stop the stack?

Not with `docker compose down` — the ledger and the database live on named volumes and survive. Use `docker compose down -v` when you deliberately want a clean slate.

Can I run the indexer twice for redundancy?

It will not corrupt anything — every write is an upsert on a unique key, so a second instance duplicates work rather than data. But it also gains you nothing, because there is no partitioning or leader election. One instance is the intended deployment.

Why are slots and fees strings in the JSON?

Because they are 64-bit integers and JavaScript's `Number` cannot hold all of them exactly. Returning them as strings guarantees no digits are lost. See the warning at the start of Chapter 5.

Why does a transaction show no compute units?

Either the validator did not report them for that transaction, or the transaction consumed exactly zero and hit a known issue (defect D-06 in the project report), where a zero value is currently stored as “unknown”. The fee and everything else are unaffected.

Can I put this on a shared server?

Only on a trusted network. There is no authentication — anyone who can reach port 3000 can read everything indexed. Adding an authentication layer is listed as future work.

How do I upgrade an existing installation?

Pull the new code and run `docker compose up --build -d`. Migrations are applied automatically at indexer start-up, and only unapplied ones run, so your existing data is preserved.

CHAPTER 10

Glossary

TERM	MEANING
Account	An addressable unit of state on Solana. Every transaction names the accounts it reads or writes.
Base-58	The text encoding Solana uses for signatures and public keys. The indexer converts raw byte buffers into it.
Commitment level	How settled a transaction is. This project subscribes at `PROCESSED`, the earliest and lowest-latency level.
Compute units	Solana's measure of execution cost, analogous to gas.
Geyser plugin	A shared library loaded into the validator that receives account, slot, transaction and block events as they happen.
gRPC	A high-performance RPC protocol using Protocol Buffers. Yellowstone uses it to stream validator events.
Instruction	One operation within a transaction, naming a program and the accounts it acts on.
Lamport	The smallest unit of SOL. 1 SOL = 1,000,000,000 lamports. All balances are stored in lamports.
Slot	A time window in which a leader may produce a block. Slot numbers increase monotonically and order the transaction list.
Signature	The base-58 identifier of a transaction — the primary way to look one up.
Upsert	Insert-or-update in one operation. Used everywhere in the indexer so that reprocessing a transaction cannot create a duplicate.
Yellowstone gRPC	The Geyser plugin, maintained by Triton One, that exposes the validator's event stream over gRPC.

WHERE TO GO NEXT

For first-time setup with per-step verification, see the **Installation Guide**. For the design rationale, data model and full test evidence, see the **Final Project Report**. For the complete test catalogue on its own, see the **Test Cases and Validation Report**.